

MANUAL DE EJERCICIO 2

Sistema Anti-Overflow (Semana 10)

1. Objetivo

Desarrollar un sistema que permita generar una secuencia de potencias a partir de un número entero ingresado por el usuario, multiplicándolo sucesivamente por sí mismo hasta detectar un posible desbordamiento de valores, simulando el control de límites en operaciones numéricas.

2. Descripción del problema

En programación, los tipos de datos numéricos tienen un límite de almacenamiento. Cuando una operación supera ese límite, ocurre un desbordamiento (overflow), lo que puede generar errores o resultados incorrectos.

Este ejercicio simula un sistema que detecta hasta qué punto un número puede ser multiplicado por sí mismo sin perder estabilidad numérica.

3. Entrada

- Número entero (N)
 - Tipo: entero
 - Puede ser positivo o negativo
-

4. Salida

- Secuencia de potencias del número ingresado
 - Último valor válido antes de un posible desbordamiento
 - Potencia máxima alcanzada
 - Mensaje de advertencia si se detecta riesgo de overflow
-

5. Proceso / Lógica

1. El usuario ingresa un número entero.

2. Se inicializa la secuencia con el número base.
3. Se multiplica el número por sí mismo de forma iterativa.
4. En cada iteración se verifica si el resultado es seguro.
5. Si el valor supera el límite permitido, se detiene el proceso.
6. Se muestra la secuencia generada y el último valor seguro.

6. Pseudocódigo

INICIO

LEER numero

SI numero es inválido ENTONCES

 MOSTRAR error

SINO

 inicializar secuencia

 potencia = 1

 resultado = numero

 MIENTRAS resultado es seguro HACER

 agregar resultado a secuencia

 potencia = potencia + 1

 resultado = resultado * numero

 SI resultado excede límite ENTONCES

 detener proceso

 FIN SI

FIN MIENTRAS

MOSTRAR secuencia

MOSTRAR último valor válido

MOSTRAR potencia máxima

FIN SI

FIN

7. Algoritmo (Java / lógica general)

```
public class AntiOverflow {  
  
    public static void generarSecuencia(int numero) {  
        long resultado = numero;  
        int potencia = 1;  
  
        System.out.println("Secuencia:");  
  
        while (true) {  
            System.out.println("Potencia " + potencia + ": " +  
resultado);  
  
            if (resultado > Integer.MAX_VALUE / Math.abs(numero)) {  
                System.out.println("Advertencia: posible overflow  
detectado");  
                break;  
            }  
            potencia++;  
            resultado *= numero;  
        }  
    }  
}
```

```
        }

        resultado = resultado * numero;
        potencia++;
    }

    System.out.println("Último valor seguro: " + resultado);
}
}
```

8. Validaciones

- El número debe ser entero válido
 - Se debe controlar el rango para evitar overflow
 - El sistema debe detener la ejecución si se detecta riesgo de desbordamiento
-

9. Ejemplo de ejecución

Entrada:

- Número: 2

Salida:

- 2
 - 4
 - 8
 - 16
 - 32
 - ...
 - Último valor seguro antes del límite
-

10. Conclusión

Este ejercicio permite comprender el concepto de desbordamiento numérico en programación y la importancia de controlar operaciones matemáticas para evitar errores en sistemas reales. Además, refuerza el uso de estructuras repetitivas y validaciones en el manejo de datos.